

ServisAku Partner Backend

Final Delivery Report — Stages 6–9 + Final Hardening

Admin Backend · Analytics · Testing & QA · Documentation · Production Hardening

1. Executive Summary

This report covers Stages 6 through 9 of the ServisAku Partner backend build: the Admin Backend with role-based access control (RBAC), the Analytics reporting layer, an automated test suite, and this documentation pass. It builds directly on Stages 1–5 (Payment Gateway, Media Uploads, Notification Dispatcher, Smart Dispatch, and Real-Time Communication), all previously delivered and live-verified against the team's shared PostgreSQL database.

Across these four stages: 73 new admin endpoints and 11 new analytics endpoints were added (84 total, on top of the roughly 60 endpoints from Stages 1–5); a pre-existing, pre-seeded RBAC schema that no prior code had ever used was wired up end-to-end; a real automated pytest suite was built from nothing, reaching 208 passing tests at 74% statement coverage; and two genuine application bugs were found and fixed during live verification. Every stage's work was pushed to the shared GitHub repository after verification, with a detailed commit history preserved throughout.

The backend remains fully additive to the shared database — zero tables were created, dropped, or redesigned across all four stages. Every new feature maps onto tables that already existed, built by other team members for modules this backend was filling in.

A subsequent Final Hardening stage raised statement coverage from 74% to 82% (307 passing tests, up from 208), fixed a codebase-wide naive-datetime timezone bug (32 files), added production rate limiting and CORS/JWT startup safety guards, added GitHub Actions CI, and produced two analysis-only deliverables (a Jobs-vs-Bookings reconciliation and a Socket.IO horizontal-scaling strategy). Writing the new payment tests surfaced a genuine, previously-undiscovered production bug — refund creation had been silently returning 500 for every caller due to a schema-drift issue — found and fixed. Zero database schema changes; zero production data deleted.

2. Backend Architecture

FastAPI (async) + SQLAlchemy 2.0 (async ORM) + PostgreSQL, structured in four layers: routes (request validation and orchestration only), services (business logic — payment gateways, Smart Dispatch, notifications, RBAC, real-time events), models (thin SQLAlchemy declarative classes), and schemas (Pydantic request/response contracts kept independent of the ORM models, so the external API shape can stay stable even when a table's real column doesn't match it 1:1).

Two integration points use a provider-agnostic pattern uniformly: payments (Billplz/iPay88 behind a common gateway interface) and notifications (push/SMS/email behind a common registry with automatic provider fallback). Real-time events flow through a minimal in-process pub/sub bus (services/realtime/events.py) so business logic never imports the Socket.IO layer directly — this avoids a circular import and keeps the real-time transport swappable.

The RBAC layer added in Stage 6 (services/rbac.py) sits on top of the existing coarse role check (partner/consumer/admin from the JWT) as a second, granular layer: `require_permission("partners.approve")` resolves a user's effective permissions via `user_roles` → `roles` → `role_permissions` → `permissions` — tables that existed and were pre-seeded before this project touched them, but had never been queried by any code.

3. Database Architecture

A single shared PostgreSQL database (servisakudb, AWS RDS, PostGIS-enabled), built up incrementally across many team members' modules — 83 tables as of the most recent full schema query, stable since Stage 4. This backend's discipline throughout every stage: query the live schema first, map to what actually exists, never assume or redesign.

- PostGIS geography columns (partners.home_location, consumer_addresses.location) power Smart Dispatch's proximity search via raw parameterized SQL (ST_Distance/ST_DWithin), not ORM-mapped.
- Every enum is Postgres-native (create_type=False) — SQLAlchemy never tries to recreate an existing type.
- Identity is centralized in a shared users table, the FK target for every module's user-linked rows.
- job_dispatches doubles as both the live offer queue and the permanent dispatch/assignment log — rows are never deleted, only status-transitioned.
- A generated column (audit_logs.retention_until) was discovered the hard way in Stage 6 — documented below under Testing Summary.
- No analytics views or materialized views exist in the schema — confirmed before Stage 7 — so every analytics endpoint computes live.

4. API Documentation

Full interactive documentation is served at /docs (Swagger UI) and /redoc, generated from FastAPI's OpenAPI schema — every endpoint has a description, permission requirement, and example request/response. 25 tag groups organize roughly 144 REST paths (170+ operations counting each HTTP method separately). API_TESTING_REPORT.md is the manually-verified companion checklist, tracking pass/fail status and notes per endpoint across every stage.

Stage	Endpoint groups added	Endpoint count
1–3	Consumer, Payments, Uploads, Notification Dispatch	39
4	Smart Dispatch	11
5	Chat (+ Socket.IO events)	4
6	Admin (10 groups: Dashboard, RBAC, Users, Partners, Bookings, Catalog, Coupons, Settlements, Support, Training)	73
7	Analytics	11

5. Authentication Flow

POST /auth/login (phone + password) or the OTP verification flow issues a short-lived JWT access token (15 min default) and a longer-lived refresh token (30 days default), each carrying a type claim so one can never be substituted for the other. Every protected route resolves the caller's own scope ID (partner_id/consumer_id/user_id) server-side from the token's sub claim — never from a client-supplied parameter — so cross-account access is structurally prevented, not just filtered after the fact. Stage 6

added a second authorization layer (RBAC) on top of this for admin endpoints, detailed in the Backend Architecture section above.

6. Payment Flow

Consumer creates a booking (PENDING_PAYMENT) → POST /payments/bookings/{id}/bill creates a gateway checkout bill (Billplz live/self-serve sandbox; iPay88 stubbed pending merchant credentials) behind a provider-agnostic gateway interface — swapping providers never touches route or business-logic code. The gateway's webhook (POST /payments/billplz/callback) verifies an HMAC signature before trusting any status change, then moves the booking to CONFIRMED and automatically triggers Smart Dispatch. Payments sit in HELD_IN_ESCROW until explicitly released by an admin; refunds follow a full approval workflow (REQUESTED → PENDING_APPROVAL → APPROVED → PROCESSING → COMPLETED/REJECTED/FAILED), every admin decision now logged to the Stage 6 audit trail.

7. Upload Flow

Avatar, KYC document, and job-photo uploads via Cloudinary's free tier, with two paths: a validated server-side upload (simplest) and a signed direct-to-Cloudinary upload for large files on slow connections. Every upload is validated by real magic-byte content sniffing (JPEG/PNG/WEBP signatures), not the client-supplied Content-Type header, which is trivially spoofable — a renamed .txt file is rejected regardless of its declared type. Stage 6 added admin-side KYC document review (verify/reject) on top of the existing partner-side upload flow.

8. Dispatch Flow (Smart Dispatch, Stage 4)

Triggered automatically when a booking's payment is confirmed. A candidate pipeline (services/dispatch/matching.py) finds nearby ACTIVE, available partners via PostGIS within their own service radius, filters by availability slot, skill match, blocked-match exclusion, and daily workload cap, then scores the remainder on a weighted formula (proximity 35%, rating 25%, completion rate 20%, language match 15%, workload 5%). Offers are sequential, not broadcast — exactly one PENDING job_dispatches row exists per booking at a time, with a configurable acceptance timeout, automatic retry to the next-ranked candidate on decline or expiry, and a background sweep worker handling expiries independently of any active request. Manual admin override and full dispatch/assignment history are exposed via the API.

9. Notification Architecture

A central dispatcher (services/notifications/dispatcher.py) always creates the in-app notification row, then best-effort delivers on every other requested channel (push via Firebase, email via a Resend → Brevo → MailerSend fallback chain, SMS via a mock provider pending a real SMS vendor), honoring per-user, per-category channel preferences and logging every delivery attempt — including "no provider configured" — to a dedicated delivery log table. A failing or unconfigured channel never breaks the business action that triggered it (registration, booking confirmation, payment, feedback); this was verified live and one real bug (a background-task database-session race) was found and fixed during Stage 3 development.

10. Socket.IO Architecture (Stage 5)

A JWT-authenticated python-socketio AsyncServer mounted alongside the REST API (uvicorn main:socket_app). Clients join role-scoped rooms (user:{id}, partner:{id}, consumer:{id}, booking:{id}, the last permission-checked per-join) and receive live chat messages, typing indicators, read receipts, partner location updates, presence, heartbeat, and — via the same in-process event bus business logic already emits to — dispatch job offers and booking status updates in real time. Presence and partner GPS location are intentionally not persisted (no dedicated table exists; both are inherently ephemeral). A pre-existing timezone bug (asyncpg silently misinterpreting naive datetimes) and a structlog logging kwarg collision were both found during this stage's live verification; the timezone fix was scoped to this stage's own new code at the time and completed codebase-wide during Final Hardening (see Executive Summary).

11. Analytics Overview (Stage 7)

11 read-only endpoints under /admin/analytics/, gated by the Stage 6 RBAC reports.read permission: revenue, bookings, partner performance, consumers, trend metrics (a time-series companion to the Stage 6 dashboard's point-in-time snapshot), conversion funnel, cancellations, dispatch (a thin alias for the existing Stage 4 endpoint — verified byte-for-byte identical, not a duplicated implementation), payments, notifications, and support. No analytics view or materialized view exists in the live schema — confirmed before writing any code — so every number is computed live via aggregate SQL against tables already mapped in earlier stages.

12. Testing Summary (Stage 8)

A real automated pytest suite was built from scratch (none existed before Stage 8): unit tests (password hashing, JWT round-trips, upload MIME sniffing, KYC status mapping — no DB or network), API tests across every route file from every stage, genuine integration tests (a support-ticket lifecycle, a catalog CRUD lifecycle, an RBAC role-assignment lifecycle, and an end-to-end Smart Dispatch flow exercising real candidate ranking and the full start/decline/retry/accept engine path against the live database), and Socket.IO connection tests.

Metric	Stage 8 result	Final Hardening result
Tests passing	208 / 208	307 / 307
Tests failing	0	0
Statement coverage (application code)	74%	82%
Real application bugs found	0	1 (refund creation 500 — see below)
CI coverage gate	none	80% (.github/workflows/backend-ci.yml)

Final Hardening raised coverage by targeting the largest remaining gaps directly: notification-dispatcher retry/fallback orchestration (tested against a mocked provider boundary, clearly distinguished from real-provider tests), every third-party integration's real "not configured" error path (Firebase, Billplz, iPay88, Cloudinary, Resend/Brevo/MailerSend — unmocked, exercising the actual provider code), RBAC permission resolution, Smart Dispatch retry/exhaustion paths, the complete payment/refund state machine, and upload success paths. No modules were excluded from the coverage denominator to inflate the number, and the CI gate was not set below 80% to make it easier to pass.

Writing the payment-lifecycle tests surfaced a genuine, previously-undiscovered production bug: `refunds.requires_approval` had become a Postgres `GENERATED ALWAYS` column in the live schema (the same class of issue as the Stage 6 `audit_logs.retention_until_discovery`), but the ORM still mapped it as writable — meaning `POST /payments/{id}/refunds` had been returning a raw 500 for every caller. Found and fixed; the full refund lifecycle (request, approve, reject, complete, partial refunds) is now verified end-to-end. A `Socket.IO` test flakiness issue was also root-caused (a client-side connection-timeout default too aggressive for a handler doing real DB round trips, not an application bug) and fixed properly rather than papered over with longer waits — verified stable across multiple consecutive full-suite runs.

The suite runs against the real shared development database — no separate test database is provisioned for this project. This is a deliberate, documented tradeoff (`docs/TESTING_GUIDE.md`): it caught three real schema-shape/mapping bugs across the project's lifetime that a mocked database would have missed, at the cost of tests not being isolated from concurrent manual/team activity. Every data-creating test uses unique, timestamp-suffixed identifiers and is written to be safe to re-run indefinitely.

13. Security Review

JWT authentication (bcrypt-hashed passwords, separate access/refresh token types), two-layer authorization (coarse role check + Stage 6's granular RBAC, verified to fail closed for a zero-permission role), server-resolved ownership checks throughout (never client-supplied), Pydantic-validated input on every request body, magic-byte content validation on every file upload, and parameterized SQL everywhere (including the raw-SQL PostGIS and analytics queries). A full audit trail (`admin_actions` + `audit_logs`) covers every admin mutation across every stage, not just Stage 6's own new endpoints.

Final Hardening added two structural safety mechanisms. First, production rate limiting (`slowapi`, in-memory by default and Redis-ready) on login, OTP request/verify, payment creation, all four refund endpoints, all three upload endpoints, notification broadcast, and seven admin-sensitive endpoints, returning 429 with `Retry-After` headers once a client-IP limit is exceeded. Second, startup-time production guards in `config.py`: the application now refuses to boot at all if `ENVIRONMENT=production` and either `ALLOWED_ORIGINS` is still the wildcard `"*"` or `JWT_SECRET_KEY` is still the placeholder default — turning what used to be a documentation note someone had to remember into a failure that cannot be silently missed. The Billplz webhook's HMAC signature verification (`hmac.compare_digest`, constant-time) and Cloudinary's magic-byte upload validation were both re-verified this stage and confirmed to already fail closed.

Known gaps, disclosed rather than hidden:

- No CSP/security headers middleware — acceptable for an API-only backend with no direct HTML rendering; add secure-headers-style middleware if an admin web dashboard is ever built to consume this API directly.
- No JWT secret rotation / per-token revocation mechanism.
- Rate limiting is in-memory and single-process — correct for the current single-worker deployment; needs Redis-backed storage before running multiple `uvicorn` workers (same caveat class as `Socket.IO`'s in-process state, see `Socket.IO Scaling` below).
- `get_remote_address` (the rate limiter's client-IP key function) trusts `request.client.host` directly — behind a reverse proxy, `X-Forwarded-For` trust must be configured correctly or every request shares one bucket.

- Billplz, iPay88, Cloudinary, Firebase, and every email provider remain unverified against real sandbox/live credentials — every "not configured" error path is tested, but no real outbound call has ever succeeded in this environment.

Full detail in docs/SECURITY.md.

14. Deployment Instructions

No production deployment target exists yet — development and verification have run against the shared RDS instance via an SSH tunnel from a local machine. The real ASGI entrypoint for production is `uvicorn main:socket_app` (not `main:app`, which drops real-time functionality). A known scaling caveat: the real-time event bus is in-process, so multi-worker/horizontal scaling needs either a Redis-backed Socket.IO message queue or sticky sessions before it can be safely load-balanced. Full checklist — environment variables, TLS termination, background-worker behavior, and recommended next steps — is in `docs/DEPLOYMENT.md`.

15. Jobs vs Bookings Analysis (Final Hardening)

Analysis-only deliverable, per explicit instruction — no code, schema, or data changes. Documented in full at `docs/JOBS_BOOKINGS_RECONCILIATION.md`. Finding: this backend has two parallel, disconnected representations of a unit of paid work. The jobs table (with its own customers/earnings/settlements satellite tables) is populated only by `seed.py` — no route, service, or webhook has ever created a new Job row. The bookings table is the live, actively-developed domain every Stage 1–9 feature (payments, Smart Dispatch, chat, admin, analytics) operates on. Most consequentially: no code path anywhere creates an Earning row when a booking completes, meaning a partner's wallet/settlement history today reflects only seed data regardless of how many real bookings they complete. A phased, additive migration strategy is documented, starting with closing exactly that gap — but nothing was implemented this stage, per instruction.

16. Socket.IO Scaling Strategy (Final Hardening)

Documentation only, per explicit instruction — no Redis or other infrastructure was deployed. Documented in full at `docs/SOCKET_SCALING.md`. The current single-process architecture holds three separate pieces of state entirely in memory (session/presence tracking, socket.io's own room manager, and the in-process business-event pub/sub bus), none of which is visible across processes — running multiple uvicorn workers today would silently drop real-time events delivered to a socket connected to a different worker than the one that triggered them. The documented fix is python-socketio's built-in `AsyncRedisManager`, which requires no application-logic changes beyond swapping the client manager, plus an ordered, reversible migration plan (provision Redis, add a feature-gated setting, load-test with two workers before scaling further).

17. Continuous Integration (Final Hardening)

`.github/workflows/backend-ci.yml` runs on every pull request and push to main touching `backend/**`. An import-check job always runs (installs dependencies, imports the app with a placeholder database URL — no live database needed, since SQLAlchemy's async engine connects lazily) and needs no secrets. A second job runs the full pytest suite with an enforced 80% statement-coverage gate, but requires a `DATABASE_URL` repository secret pointing at a real, already-seeded PostgreSQL instance — this test

suite has no separate test database by deliberate project design (see Testing Summary above), and that decision carries through to CI. The workflow fails loudly with an actionable message if that secret isn't configured, rather than silently skipping and looking like a pass.

18. Git Commit History

Pushed to origin/main at github.com/Dineshkuppuraj17/servisaku-partner-consumer. Each stage is one feature commit plus a short docs-update commit recording its own hash in docs/today-work/GIT_COMMITS.md — see that file for the complete history including Stages 1–5, the repository migration, and this Final Hardening stage's exact commit hashes (recorded there immediately after push, following the same pattern used for every prior stage).

Commit	Description
c07698f	feat(admin): Stage 6 Admin Backend — RBAC, catalog, partner approval, ops
326d9a2	docs: record Stage 6 commit hash
8e53f32	feat(analytics): Stage 7 Analytics — 11 read-only reporting endpoints
109da90	docs: record Stage 7 commit hash
7085a82	test(qa): Stage 8 Testing & Quality Assurance — pytest suite, 208 tests
bda3615	docs: record Stage 8 commit hash
3e36b15	docs: Stage 9 Documentation & Final Delivery
cd67ecc	docs: record Stage 9 commit hash
(see GIT_COMMITS.md)	Final Hardening: timezone fix, rate limiting, production guards, CI, coverage 74%→82%, Jobs/Bookings + Socket.IO scaling analysis

19. Future Improvements

- Obtain real Billplz, Cloudinary, Firebase, and email provider credentials — the single biggest remaining gap, blocking full end-to-end verification of those integrations. Still true after Final Hardening.
- Configure the DATABASE_URL (and, if needed, SSH tunnel) repository secrets so the new CI workflow's test-and-coverage job actually runs.
- Close the jobs-vs-bookings payout gap identified in the Final Hardening analysis: add a booking-keyed Earning creation path so partners are paid out for real Smart-Dispatch-assigned bookings, not just seed data — see docs/JOBS_BOOKINGS_RECONCILIATION.md for the recommended phased approach.
- Provision Redis and wire up AsyncRedisManager before deploying more than one uvicorn worker — see docs/SOCKET_SCALING.md for the ordered migration steps; also apply RATE_LIMIT_STORAGE_URI=redis://... at the same time, since rate limiting has the identical in-process-state caveat.

- Add JWT secret rotation / a per-token revocation mechanism before public launch.
- Consider a per-role permission audit for the pre-seeded RBAC data — the READ_ONLY role currently has zero granted permissions, which may or may not be the intended design.
- Build out the deferred Stage 6/7 partial features once real usage data exists: training-progress write path for partners, subscription/package creation flow, surge-pricing-rule activation logic.

This concludes the Stage 6–9 + Final Hardening delivery. All work is committed and pushed to origin/main; no further implementation was started pending review.